

ACO Edge Detection

Code Documentation — `aco_edge.cpp`

1 What the Program Does

`aco_edge` takes any image file as input, runs an Ant Colony Optimization (ACO) algorithm to find edges, and displays the result in a live window using the TIGR library. The window shows the grayscale original on the left and the evolving edge map on the right, updating after every iteration. When finished, the result is saved as `<input>_edges.png`.

```
1 ./aco_edge <input_image>
```

2 Parameters

Constant	Value	Role
NUM_ANTS	8000	ants released each iteration
NUM_ITERATIONS	30	outer loop count
ANT_STEPS	50	steps each ant walks
ALPHA	1.8	pheromone influence exponent
BETA	1.2	heuristic influence exponent
RHO	0.1	evaporation rate
Q	1.0	pheromone deposit scale
TAU0	0.1	initial pheromone value
TAU_MIN	0.01	Min-Max lower clamp
TAU_MAX	9.0	Min-Max upper clamp
EDGE_THRESHOLD	0.35	fraction of <code>TAU_MAX</code> to classify as edge

3 Algorithm

3.1 Preprocessing

Grayscale conversion. Each pixel is converted from RGB to a single intensity value:

$$I_{ij} = 0.299 R + 0.587 G + 0.114 B$$

Heuristic η_{ij} (Sobel gradient magnitude). For every pixel the horizontal and vertical gradients are computed over a 3×3 neighbourhood and combined:

$$G_x = \sum_{\text{Sobel}_x}, \quad G_y = \sum_{\text{Sobel}_y}, \quad \eta_{ij} = \sqrt{G_x^2 + G_y^2}$$

All values are normalised to $[0, 1]$ by dividing by the maximum. Pixels on strong edges get values close to 1; flat regions get values close to 0.

3.2 Ant Movement

Each ant starts at a random pixel. At every step it chooses one of its 8-connected neighbours probabilistically. Moving straight back to the previous pixel (180-degree backtracking) is forbidden.

The probability of moving from (l, m) to neighbour (i, j) is:

$$P_{(l,m) \rightarrow (i,j)} = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{(s,q) \in \Omega(l,m)} \tau_{sq}^\alpha \eta_{sq}^\beta}$$

- τ_{ij} is the pheromone — how much collective evidence there is that (i, j) is on an edge.
- η_{ij} is the gradient heuristic — the local visual evidence of an edge at (i, j) .
- $\alpha = 1.8$ weights pheromone; $\beta = 1.2$ weights the gradient. Both are moderate, so ants balance exploiting known edges and exploring new ones.

If the total probability is essentially zero (completely flat patch), the ant picks a random direction instead of getting stuck.

When an ant visits (i, j) it deposits:

$$\Delta\tau_{ij} = Q \cdot \eta_{ij}$$

so ants leave more pheromone on strongly-edged pixels.

3.3 Pheromone Update (Max-Min Ant System)

After all ants finish their walks, pheromone is updated globally:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_{\text{threads}} \Delta\tau_{ij}$$

Then each value is clamped:

$$\tau_{ij} \leftarrow \min(\text{TAU_MAX}, \max(\text{TAU_MIN}, \tau_{ij}))$$

The Min-Max clamping prevents any single pixel's pheromone from dominating (which would collapse all ants onto one path) and stops evaporation from reducing pheromone all the way to zero.

3.4 Edge Classification

After normalising by TAU_MAX, a pixel is drawn as an edge if:

$$\frac{\tau_{ij}}{\text{TAU_MAX}} \geq \text{EDGE_THRESHOLD}$$

Edge pixels are coloured with a gradient (deep blue $\rightarrow$ neon pink $\rightarrow$ cyan) proportional to edge strength. Non-edge pixels are shown as a dimmed version of the original grayscale (30% brightness).

4 Pseudocode

Algorithm 1 ACO Edge Detection with TIGR display

```

1: Load image; downscale to max dimension 800 if needed
2: Convert to grayscale  $I$ ; compute Sobel heuristic  $\eta$ 
3:  $\tau_{ij} \leftarrow \tau_0$  for all pixels
4: Open TIGR window; draw grayscale on left half
5: for iter = 1 to NUM_ITERATIONS do
6:    $\Delta \leftarrow 0$  for all pixels ▷ per-thread buffers
7:   for  $k = 1$  to NUM_ANTS do ▷ #pragma omp parallel for
8:     Place ant at random pixel  $(l, m)$ ; last  $\leftarrow (-1, -1)$ 
9:     for  $s = 1$  to ANT_STEPS do
10:      for each neighbour  $(i, j) \in \Omega(l, m)$  do
11:        if out of bounds or  $(i, j) = \text{last}$  then
12:           $p_{ij} \leftarrow 0$ 
13:        else
14:           $p_{ij} \leftarrow \tau_{ij}^\alpha \cdot \eta_{ij}^\beta$ 
15:        end if
16:      end for
17:      if  $\sum p = 0$  then
18:        choose random neighbour
19:      else
20:        choose  $(i^*, j^*)$  by roulette-wheel on  $\{p_{ij}\}$ 
21:      end if
22:       $\Delta_{i^*j^*} += Q \cdot \eta_{i^*j^*}$ 
23:      last  $\leftarrow (l, m)$ ;  $(l, m) \leftarrow (i^*, j^*)$ 
24:    end for
25:  end for
26:  for each pixel  $(i, j)$  do ▷ merge + evaporate + clamp
27:     $\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_t \Delta_{ij}^{(t)}$ 
28:     $\tau_{ij} \leftarrow \min(\text{TAU\_MAX}, \max(\text{TAU\_MIN}, \tau_{ij}))$ 
29:  end for
30:  for each pixel  $(i, j)$  do ▷ update TIGR window right half
31:    if  $\tau_{ij}/\text{TAU\_MAX} \geq \text{EDGE\_THRESHOLD}$  then
32:      draw with gradient colour (strength-based)
33:    else
34:      draw dimmed grayscale (30%)
35:    end if
36:  end for
37:  tigrUpdate(screen)
38: end for
39: Save right half as <input>_edges.png
40: Wait for window close or ESC

```

5 Parallelisation

The inner ant loop is parallelised with one pragma:

```

1 #pragma omp parallel for schedule(dynamic, 64)
2 for (int ant = 0; ant < NUM_ANTS; ant++) { ... }

```

Each thread has its own `delta` buffer, so no locking is needed during ant traversal. Threads write to different entries and the buffers are summed into `tau` after the parallel section. Each ant seeds its own `std::mt19937` from a combination of the ant index, thread id, and iteration number, making the random choices thread-safe and varied across runs.

6 Key Functions

Function	Purpose
<code>resize_image(src, 800)</code>	nearest-neighbour downscale to fit window
<code>to_gray(src, gray)</code>	RGB $\rightarrow$ float luminance array
<code>build_heuristic(..., eta)</code>	Sobel magnitude, normalised to $[0, 1]$
<code>get_edge_color(t, thr)</code>	maps edge strength to blue-pink-cyan colour

7 Build

TIGR is a single-header library (`tigr.h` + `tigr.c`). Link against the platform graphics libraries and OpenMP:

```

1 # Linux
2 g++ -O2 -fopenmp aco_edge.cpp tigr.c -o aco_edge -lGL -lX11 -lm
3
4 # Windows (MSVC)
5 cl /O2 /openmp aco_edge.cpp tigr.c /link opengl32.lib gdi32.lib

```